

基于 RT-Thread 与纳芯微的单级式多端口功率流控型无刷电机驱动

摘要

本设计提供了一个高转换效率、高控制粒度，且具有直流端口间功率分配能力的 NPC 型单级式三电平逆变器拓扑与结合 RT-Thread 实时操作系统内核的控制策略，能够提供灵活的功率分配能力与高效的实时响应，且提高了控制算法的复用与移植能力。

在现代电机驱动应用领域，传统双直流端口逆变器在面对直流端口间的功率分配问题时，需要引入额外的功率变换级来实现直流端口间的功率交换，以及在控制直流端口对交流侧的功率输出时需要进行同步与通信协调，这些问题直接影响了功率转换效率、元件成本及系统实时性。

而在软件设计策略上，利用传统 RTOS 常常难以解决多人团队在多模块的并发开发问题，以及在开发过程中对 RTOS 性能开销未知，多线程间开发常常耦合度较高，这些问题也显著影响了开发效率与测试难度。

在硬件部分，基于这两个问题，本团队提出了单级式三电平逆变器拓扑，在单个功率转换级内通过 PDSVM (Power-distribution Space Vector Modulation)控制策略，来实现直流端口间的功率交换与直流交流端口间的功率传递。

在软件部分，团队借助 RT-Thread 高性能 RTOS 内核，来实现 MCU 单核运行时的高性能多线程处理，同步管理控制系统中的多个硬件外设，如串口、显示与交互、功能切换、功率需求调整等等内容。并进一步地利用 RT-Thread 的组件化构建特性，实现并行多线开发，最后通过宏函数导入系统，实现无感更新，提升开发效率。在测试时，利用 RT-Thread 团队的 RT-Trace 工具能够以 ns 级单位来监视系统在运行时的性能开销，精确定位需要优化的线程所在。在开发结束后，借助 RT-Thread 的规范化 HAL 层接口，可以将工程设置高效地实现轻量级抽象，便于后续二次开发与移植适配。

同时，本团队也在应用并实现高性能 NS800RT7 系列 MCU 进行工程实

现时，同步为 RT-Thread 社区进行了开源建设，提交了该系列芯片的 ADC 外设 PR，进一步优化了 RT-Thread 内核及其开发表现。

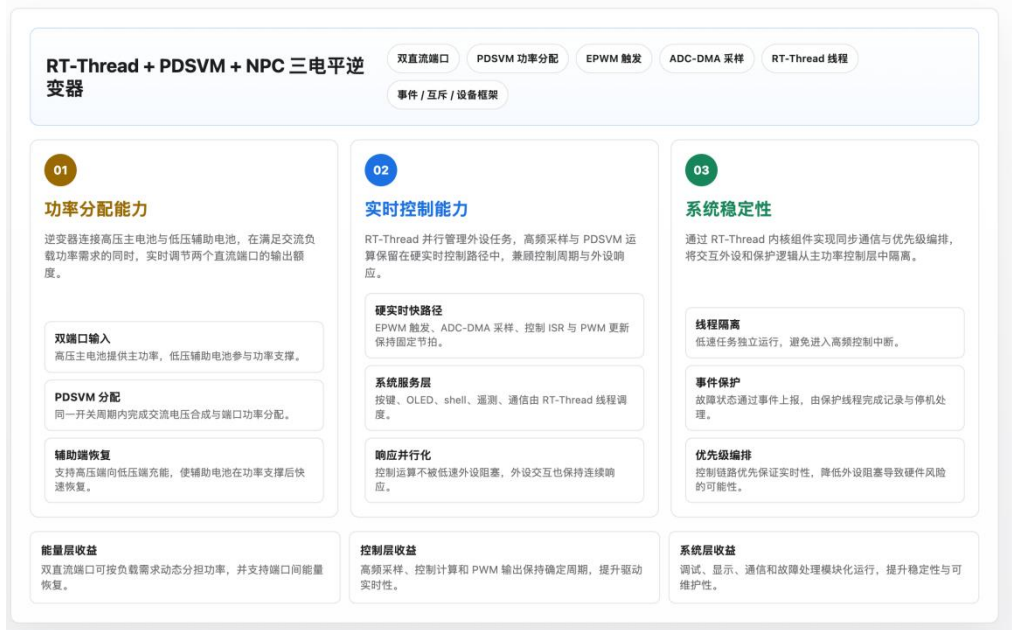
综上，本团队利用高性能 RT-Thread 内核与单级式多端口逆变器硬件设计，实现了高转换效率、具有灵活功率分配能力的三相无刷电机驱动，极大地提升了三相电机驱动系统中的功率控制能力与实时性。

第一部分 作品概述

1.1 功能与特性

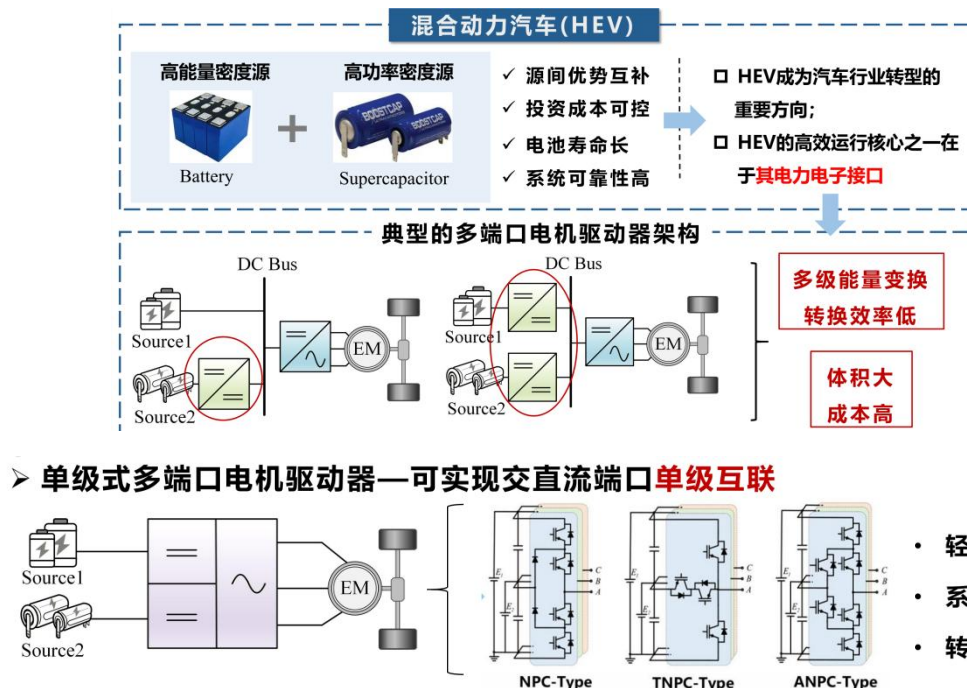
系统功能特性

本系统将单级多端口功率拓扑与 RT-Thread 实时软件架构结合，形成“功率可分配、控制可实时、系统可稳定运行”的电机驱动平台。



- 1) 功率分配能力：该逆变器与两个独立的直流电流源连接（高压主电池与低压辅助电池），能够在满足交流负载功率需求前提下，实时控制两个电流源的功率输出额度，并且能够令高压主电池为低压辅助电池进行充能，保证在低压辅助电池进行功率辅助后快速恢复。
- 2) 实时控制能力：利用 RT-Thread 内核并行处理多个外设负载，以及高频的 ADC 采样与 PDSVM 运算, 在保证运算实时性的同时保证其余外设的流畅度与响应能力。
- 3) 系统稳定性：利用 RT-Thread 内核组件，实现线程间同步通信，以及优先级编排，避免交互外设影响主功率控制层，导致系统故障甚至硬件损坏。

1.2 应用领域



本系统面向多能源供电条件下的三相无刷电机驱动与逆变功率输出场景，适用于轻型电动车、电动工具、机器人关节、无人平台动力系统及储能型交流输出设备。此类场景通常同时包含主动力电池、辅助电池、传感器与通信单元，负载功率变化快，且要求现场调参、故障保护和运行状态可观测。传统裸机控制或单一逆变平台难以同时满足功率流调度与复杂外设管理需求。本设计引入 RT-Thread 后，可将电机控制、端口功率分配、交互显示、串口调试通信和故障上报组织为可扩展的实时系统，使装置不仅能完成电机驱动，也具备现场测试、状态监测、模块扩展和后续移植能力，适合作为多端口电机驱动与混合供电功率管理平台。

1.3 主要技术特点

本系统首先采用基于 RT-Thread 的分层实时控制架构。EPWM 触发、ADC-DMA 采样、控制 ISR 与 PDSVM 运算构成硬实时快路径；按键、OLED、串口 shell、运行遥测、故障处理和通信由线程、事件、互斥量、ringbuffer 与设备框架管理，从而隔离低速外设对功率控制环的影响。

硬件方面，系统采用 NPC 型单级式三电平逆变拓扑，连接高压主电池与低压辅助电池两个直流端口，并通过 SVM 与载波生成 PWM 波形来形成三相上下桥臂 PWM 输出。PDSVM 在同一开关周期内完成交流侧电压合成与双直流端口功率分配，使高压端、低压端和三相电机之间无需额外 DC/DC 级即可实现能量调度，降低转换损耗、器件成本和级联系统控制延迟。

1.4 主要性能指标

指标	目标参数
线程切换	RT-Thread 1 kHz tick; 400 MHz 下切换 开销预测 1-3 us, 时间片粒度 1 ms
ADC 采样	10 通道 DMA, 约 100 kframe/s, 控制 10 kHz 有约 10 倍裕量
控制响应	SVM/FOC ISR 10 kHz, 周期 100 us; PWM 30 kHz
功率分配能力	xi 调节可使 HV/LV 承担比例连续迁移;

1.5 主要创新点

- 1) 单级式多端口硬件拓扑设计：在实现多直流端口功率调度的前提下，减少功率转换级。
- 2) 直流端口功率分配算法实现：在单级式拓扑内通过算法实现直流端口间的灵活功率分配。
- 3) RT-Thread 组件利用以保证系统稳定性：利用进程通信、事件、log 线程、guard 线程与 dump log 来保证系统稳定，以及问题可定位的能力。
- 4) 对 RT-Thread 适配 NS800 外设与提交 PR 以提升系统复用能力。

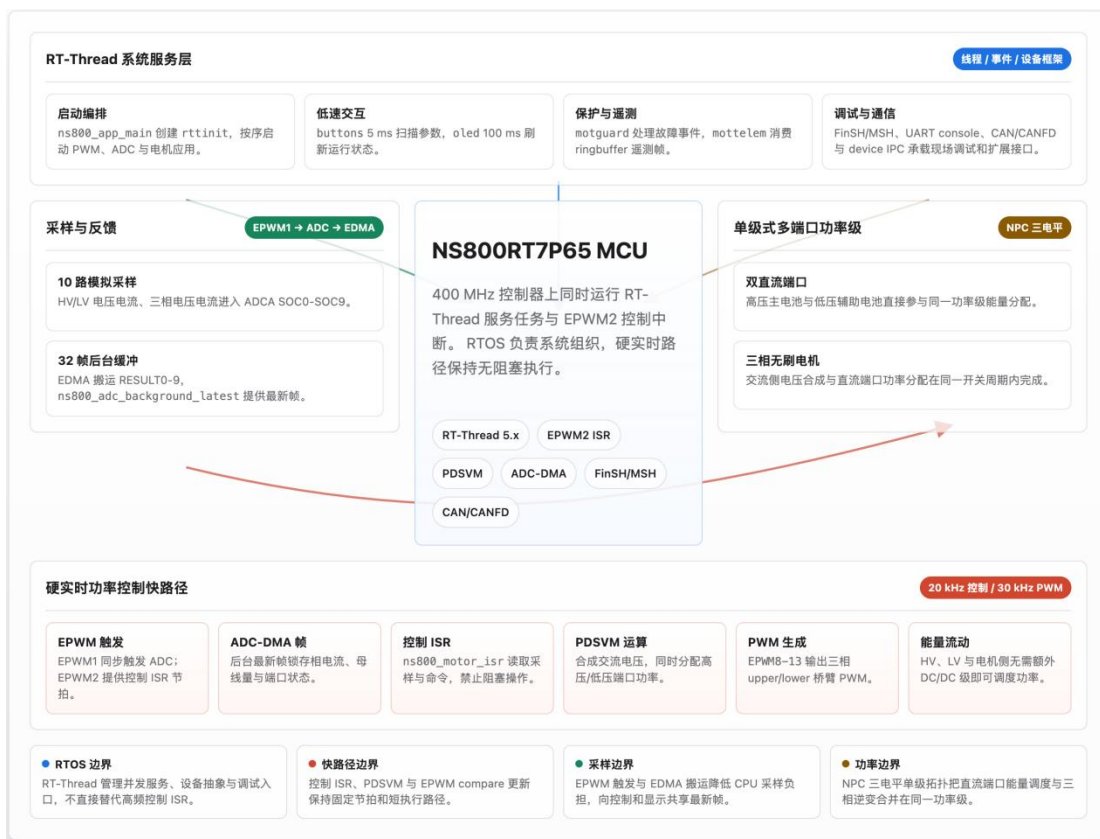
1.6 设计流程



本系统按“需求定义、软件分层、实时控制、功率分配、硬件执行”的流程设计。首先面向双直流端口与三相电机的功率流控需求，建立基于 RT-Thread 的分层软件架构，将交互、显示、调试、故障和通信等低速任务隔离在系统服务层。随后由 EPWM 触发 ADC-DMA 采样，并在控制 ISR 中完成 PDSVM 运算，实现交流电压合成与双端口功率分配，最终驱动 NPC 三电平功率级完成能量转换。

第二部分 系统组成及功能说明

2.1 整体介绍



本系统采用“RT-Thread 系统服务层 + 硬实时控制层”的软件架构。RT-Thread 负责线程调度、事件通信、互斥保护、设备抽象和调试接口，将按键、OLED、串口 shell、遥测、故障处理和通信等低速任务统一管理，避免其影响主控制环。硬实时控制层由 EPWM 触发、ADC-DMA 采样、控制 ISR、PDSVM 运算和 PWM 更新组成，直接完成采样、功率分配计算与三相桥臂驱动。该架构使系统同时具备实时控制能力、外设扩展能力和在线调试能力。

2.2 硬件系统介绍

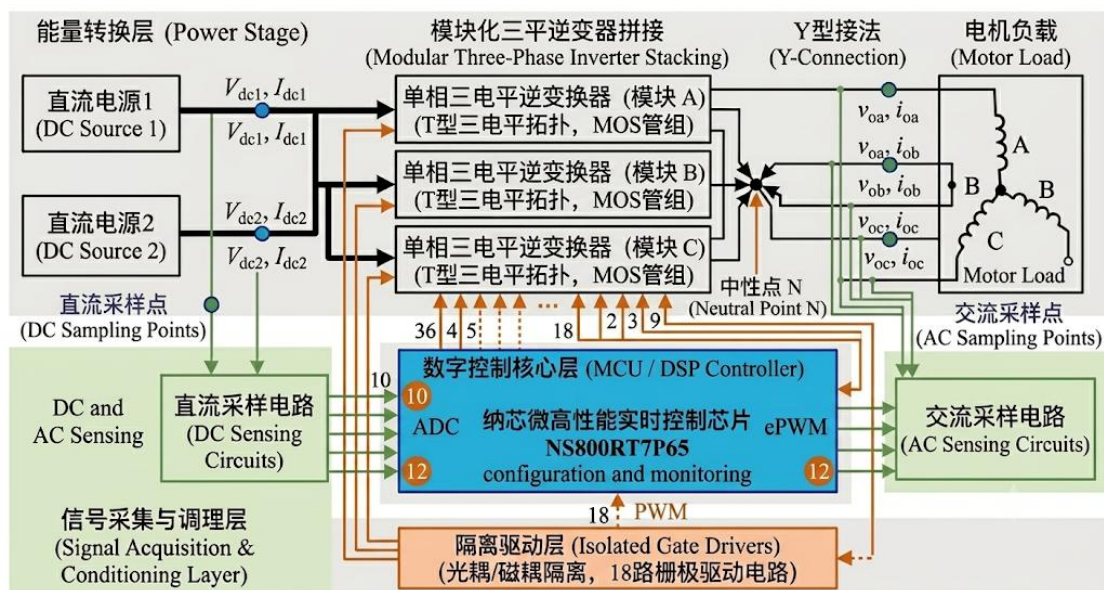
2.2.1 硬件整体介绍

本系统的硬件架构以纳芯微 NS800RT7P65 实时控制芯片为核心，主要由主功率回路、信号采样与调理电路、数字控制核心及隔离驱动电路四个部分组成，实现了强弱电的物理隔离与功率的独立调控，整体框图如图 1 所示。

在主功率回路中，系统采用两组独立的直流电源进行供电。逆变级由三个独立的三电平变换器模块组成，各模块输出端通过 Y 型接法将输入直流电逆变并驱动三相电机定子绕组。

在信号采样回路中，电压与电流传感器实时采集直流侧的 4 路输入信号以及交流侧的 6 路输出信号。传感器输出的模拟信号经过低通滤波、差分缩放和电平偏置抬升等调理后，输入至控制芯片的模数转换通道。

在控制与驱动回路中，NS800RT7P65 芯片依托内部同步 ADC 对这 10 路反馈信号进行多通道同步采样，并在内核中实时执行位置解析、双电源功率调配指令及电机双闭环矢量控制算法，计算出对应的主功率管导通占空比。生成的高频 ePWM 调制信号经过驱动芯片进行电气隔离与功率放大后，分别驱动三个逆变模块中各个 MOS 管的栅极，最终完成从信号采集、算法控制到功率输出的闭环控制。



2.2.2 电路各模块介绍

(1) 功率电路

功率电路包括三电平变换桥拓扑及其驱动电路、电源指示模块。

图 1 展示了一个标准的单桥臂中点钳位型三电平功率逆变电路。拓扑由开关器件 Q1、Q2、Q3、Q4 串联构成主桥臂，中点通过钳位二极管 D1 和 D2 连接至直流母线中点 DC2。中间的 AC 接口即为高频三电平交流电压输出端。相较于传统两电平拓扑，三电平拓扑通过引入直流分压中点和钳位二极管，能够将功率开关管承受的电压应力直接斩半。同时，高压母线输入端配备了由大容量支撑电容与高频滤波电容组成的直流母线电容阵列，降低母线寄生电感引起的开关尖峰，旁路连接了高阻值均压电阻，确保上下半桥直流母线电压的静态平衡与关断后的安全放电。

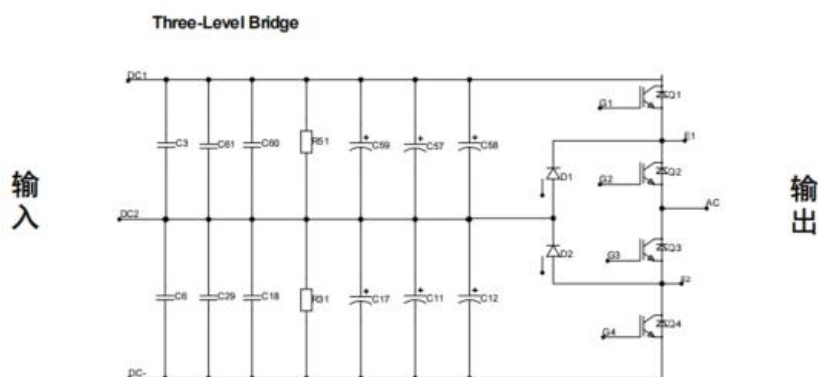


图 1 三电平功率逆变电路

由于三电平桥臂中 Q1、Q2、Q3 的源极/发射极参考电位处于高频浮动状态，电路选用了 4 组完全电气隔离的微型 DC/DC 变压模块。模块由公共的前端 AUX_15V 供电，次级输出独立的正驱动电与负偏置电压。引入负压关断是能彻底防止主回路高频 dV/dt 通过器件米勒电容耦合引发的桥臂直通误导通。驱动采用了工业级可靠带隔离保护的栅极驱动芯片，集成了 DESAT 退饱和短路微秒级保护、主动米勒钳位以及独立充放电栅极电阻调节网络，构建了高可靠性与抗干扰能力的功率变换硬件底层。

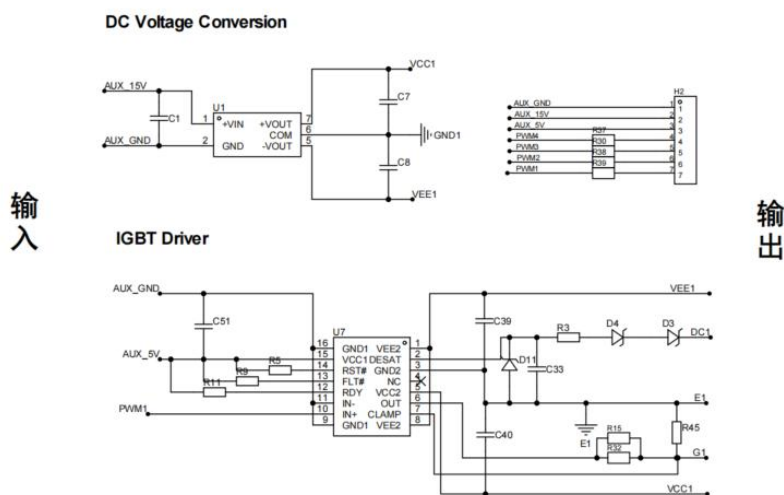


图 2 隔离驱动电源转换电路

为了确保高压实验室调试的绝对安全，电路右侧配置了由 LED 阵列。用于指示前端弱电的 AUX_15V 和 AUX_5V 状态，还能实时反映每一路次级浮动电源的工作状态，极大地方便了硬件焊接后的初次上电排查，电路设计如图 3

所示。

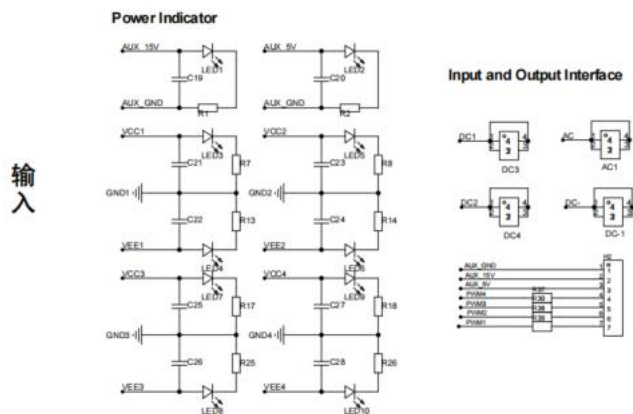


图 3 状态监控与指示模块

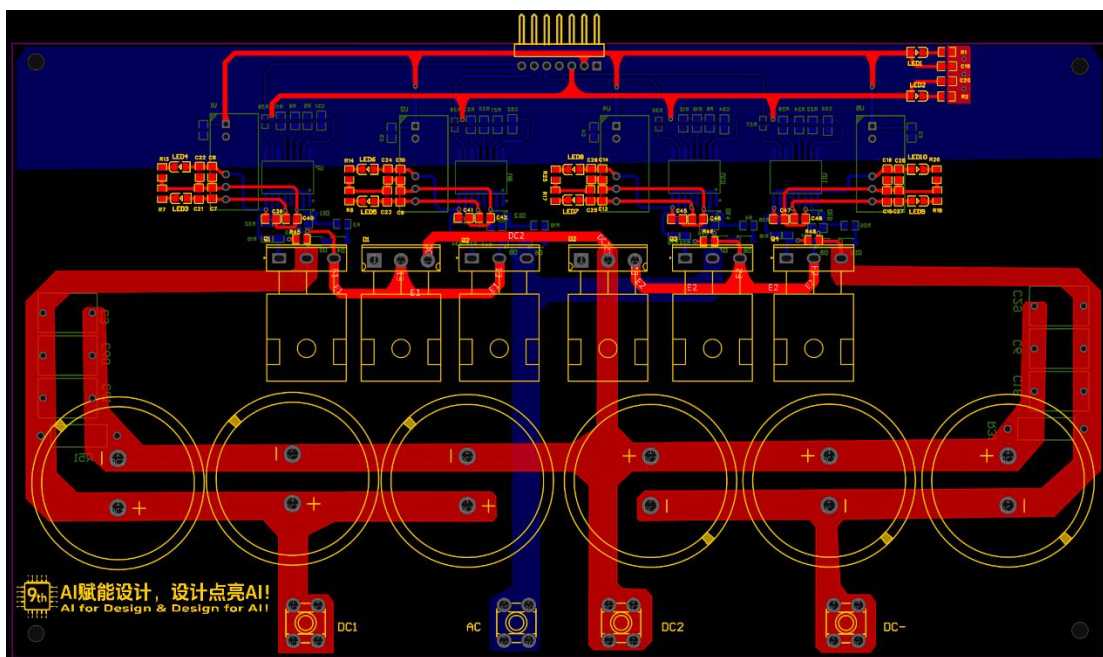


图 4 功率电路 PCB

(2) 交流采样电路

本系统设计有专用的交流电压和交流电流采样模块，用于对功率电路的输出交流信号进行实时高精度采集。由于标准单片机的模数转换器仅能采集单极性 0~3V 正电压信号，而原始交流信号在以时间为轴时存在正、负半周，因此整体信号调理电路遵循“隔离、偏置、放大、滤波”的信号调理流程。前端通过电压、电流互感器实现强弱电的电气隔离，并在次级产生微弱的交流差分信号；电路利用 REF351600DBVR 高精度电压基准芯片产生稳定的+1.6V REF 参考电压，将其

作为直流偏置注入 LM358DR 运算放大器的核心差分放大网络中，将交变信号的中点抬升至正半轴 1.6V 处；该差分放大电路在大幅抑制共模电磁干扰的同时，并在反馈通路上进行低通 RC 滤波，输出以 1.6V 为中心、在 0V 至 3.3V 安全范围内规律摆动的平滑交变电压。电路原理图与 PCB 分别如图 4、图 5 所示。

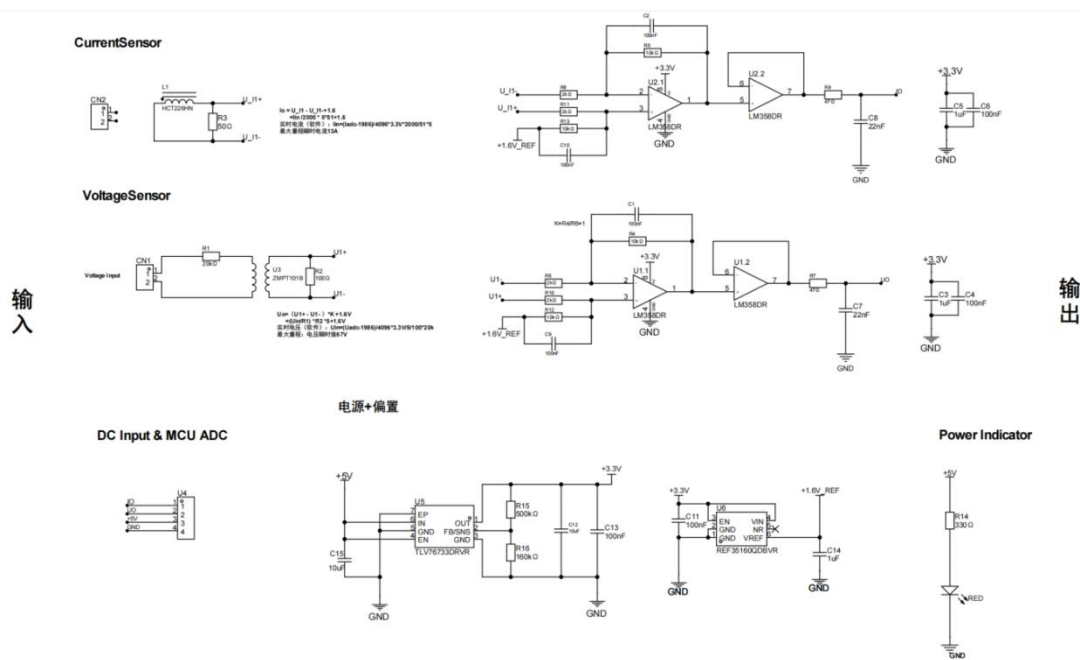


图 5 交流采样电路原理图

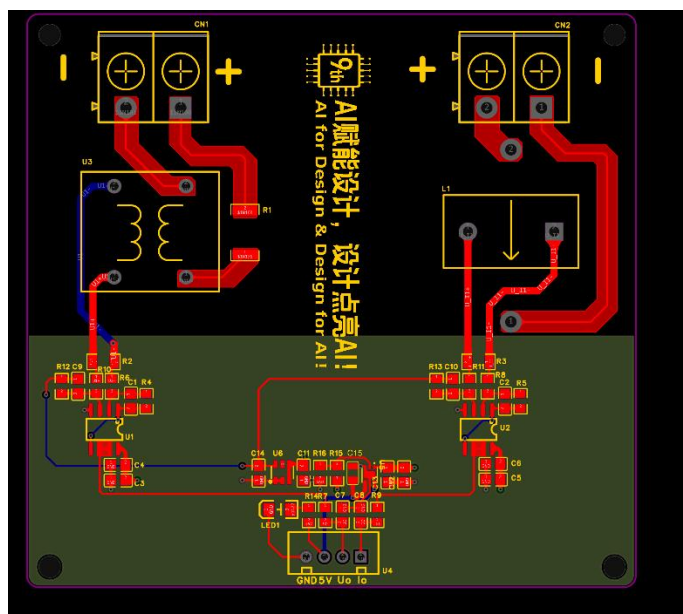


图 6 交流采样 PCB

(3) 直流采样电路

本系统设计有专用的直流电压和直流电流采样模块，重点针对单极性直流物理量设计了高阻抗、高稳态精度的调理通路。在电流采样端，被测直流通过精密采样电阻转化为微弱压降，随后进入专用电流传感放大芯片中进行增益高比例放大。在电压采样端，系统利用运算放大器组建的差分减法放大网络实现大电压比例衰减，两路信号在放大衰减完成后，均通过由运放组建的电压跟随器进行强力的阻抗匹配与驱动缓冲，并搭配 RC 低通滤波网络滤除电力电子高频开关噪声，为微控制器核心提供了稳定的 0~3.3V 直流传感信号，电路原理图、PCB 分别如图 7、图 8 所示。

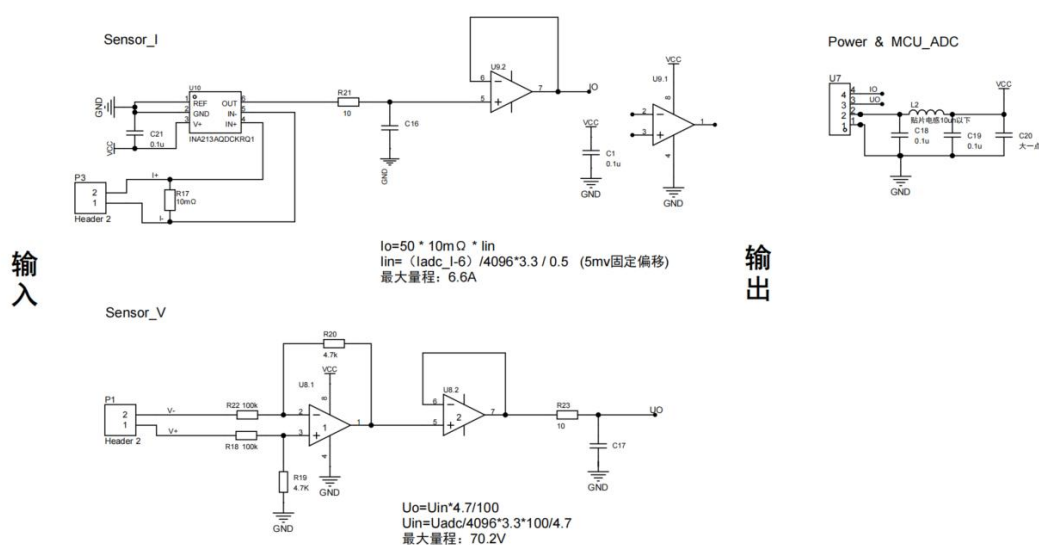


图 7 直流采样电路

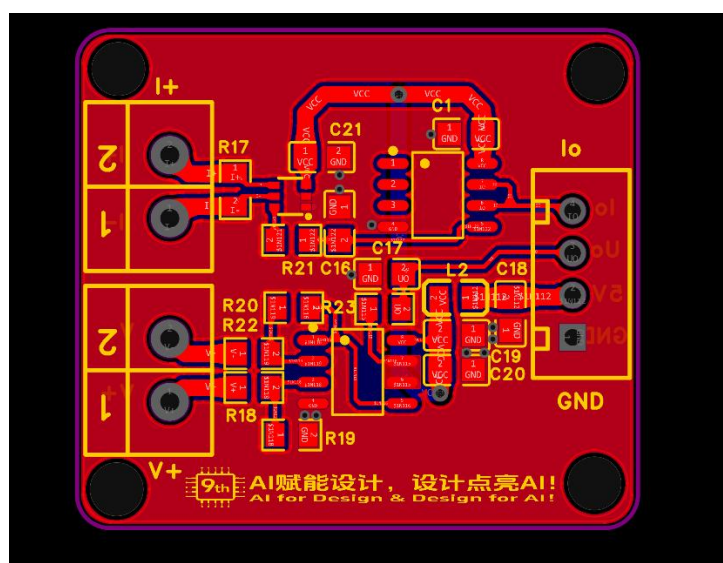


图 8 直流采样 PCB

(4) 控制电路

为满足电压、电流采样反馈以及三电平桥臂驱动等功能需求，系统设计了配套扩展板，用于完成各功能模块与主控板之间的信号连接、供电分配及电源管理。考虑到数字地噪声可能对 ADC 采样精度产生影响，ADC 采样电路采用隔离地设计，并通过单点接地方式实现模拟采样地与系统地的连接，以降低地环路干扰及数字噪声对采样结果的影响。

此外，根据系统功率分配控制需求，扩展板设置了按键切换电路，用于实现不同功率分配模式的选择；同时预留 IIC 接口以外接 OLED 显示屏，便于在调试过程中对系统运行数据进行实时观测。电路原理图、PCB 分别如图 10、图 11 所示。

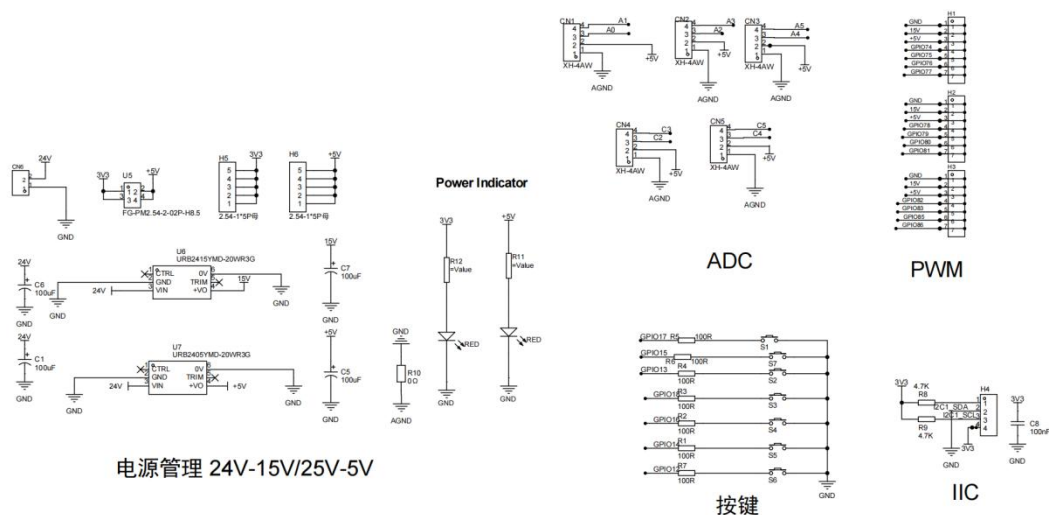


图 9 拓展板原理图

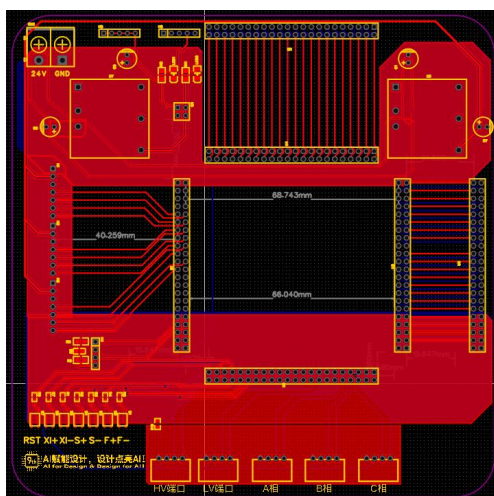


图 10 拓展板 PCB

2.3 软件系统介绍

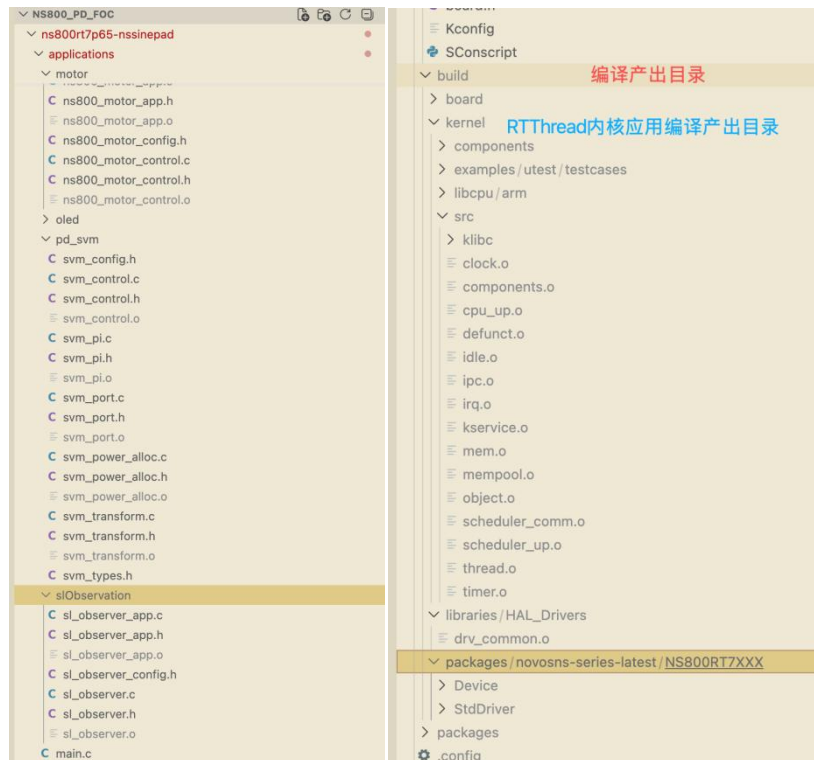
2.3.1 软件整体介绍；

整体系统分为多个模块进行开发：



Applications 目录下存放本工程的主要功能代码实现，如

- 1) 功率分配算法 PDSVM
- 2) 电机驱动适配层 motor、slObservation、Filter
- 3) 外设部分 oled、button、adc
- 4) 多线程分配与线程间通信部分*_app



2.3.2 软件各模块介绍;

1) RT-Thread 线程分配与线程同步通信:



本工程采用“RT-Thread 系统服务层 + 硬实时中断快路径”的结构。

RT-Thread 负责启动编排、线程调度、设备注册、故障处理和遥测输出；EPWM2 ISR 负责 20 kHz 电机控制, EPWM1+ADCA+EDMA 负责采样快路径。利用 RTOS 提高软件组织能力，又避免线程调度影响控制周期确定性。

初始化流程：系统上电后进入 `rtthread_startup()`。板级阶段执行 `rt_hw_board_init()`，完成时钟、SysTick、heap、pin、UART 和 console 初始化，并执行 `rt_components_board_init()`。调度器启动后创建 `main` 线程，`main_thread_entry()` 先执行 `rt_components_init()` 注册组件级设备，再调用用户 `main()`，进入 `ns800_app_main()` 创建应用线程并启动控制链路。

线程分配：应用层主要线程包括：`rttinit` 负责一次性启动 ADC-DMA 和电机控制；`buttons` 每 5 ms 扫描按键并更新 `xi/speed/F`；`oled` 每 100 ms 刷新显示；`motguard` 接收故障事件并执行保护；`mottelem` 消费遥测数据并输出日志；`tshell` 提供 FinSH/MSH 调试入口。主线程最终进入 LED2 心跳循环。

同步通信：按键到控制 ISR 采用共享参数传递，ISR 只读 `ns800_param_xi`、`ns800_param_speed`、`ns800_param_force_ma`，避免阻塞。故障使用 `rt_event` 从 ISR 通知 `motguard`。遥测使用 `rt_ringbuffer` 从 ISR 传给 `mottelem`。OLED 底层 I2C 写入使用 `rt_mutex` 保护 `i2c1` 总线。不同同步方式按实时性分层选取。

数据流架构：系统运行时形成两条并行路径：低速服务路径由 RT-Thread 线程处理按键、OLED、日志、Shell 和保护；硬实时路径由 EPWM1/ADCA/EDMA 采样、EPWM2 ISR 控制计算、EPWM8-13 PWM 更新组成。两者通过共享参数、事件和 ringbuffer 连接。RT-Thread 不替代 PWM 控制周期，而是承载外围服务和控制状态管理。

2) ADC+DMA 功能与数据同步:



ADC+DMA 模块为电机控制和状态显示提供连续采样数据。该模块不采用线

程轮询采样，而是由 EPWM1 产生 SOCA 触发 ADCA，ADCA 完成 SOC0~SOC9 转换后，通过 EDMA 自动搬运到 32 帧环形缓冲区。RT-Thread 主要负责启动编排、调试命令和多任务共享数据读取。

初始化流程： ns800_app_main() 创建 rtinit 线程，rtinit 调用 ns800_adc_background_start() 启动采样链路。初始化顺序为：停止旧 EPWM/DMA、配置 ADC 引脚、初始化 ADCA、配置 SOC 触发源、清零缓存帧、配置 DMAMUX/EDMA、启动 EDMA 传输，最后启动 EPWM1 SOCA 触发。

采样与搬运机制： ADCA 配置 10 个 SOC 通道，触发源统一为 ADC_TRIGGER_EPWM1_SOCA。ADC INT1 选择 SOC9 作为转换完成标志，并打开 ADC 到 DMA 请求。EDMA1 通道 1 通过 DMAMUX 绑定 DMAMUX_ADCA1_REQ，每次 minor loop 搬运 10 个 16 位结果，每个 major loop 覆盖 32 帧，形成无需 CPU 逐点搬运的环形采样缓存。

数据同步方式： 采样数据存放在 adc_bg_frames[32][10] 中。读取端调用 ns800_adc_background_latest()，根据 EDMA 当前写地址推算最新完整帧，并执行 DCache 失效，保证 CPU 读取到 DMA 写入后的新数据。函数还返回 seq，电机 ISR 用它判断采样帧是否更新，避免重复使用同一帧数据。

跨模块数据流： EPWM2 电机控制 ISR 读取最新 ADC 帧，其中 CH5、CH7、CH9 映射为 A/B/C 三相电流，用于控制计算。OLED 线程也读取同一最新帧，估算 HV/LV 功率和三相输出功率。FinSH 命令 adc_bg_dump 可打印当前帧，用于现场调试。该设计实现了“一路硬件采样，多任务只读共享”。

3) IIC+OLED 显示功能与线程同步：



IIC+OLED 模块承担运行状态显示功能，用于展示 HV/LV 功率、输出功率、功率分配系数 ξ 、速度给定和力矩/电流给定等信息。该模块属于低速人机交互任务，不参与 20 kHz 控制闭环，因此放在 RT-Thread 线程中周期运行，避免显示刷新影响电机控制 ISR。

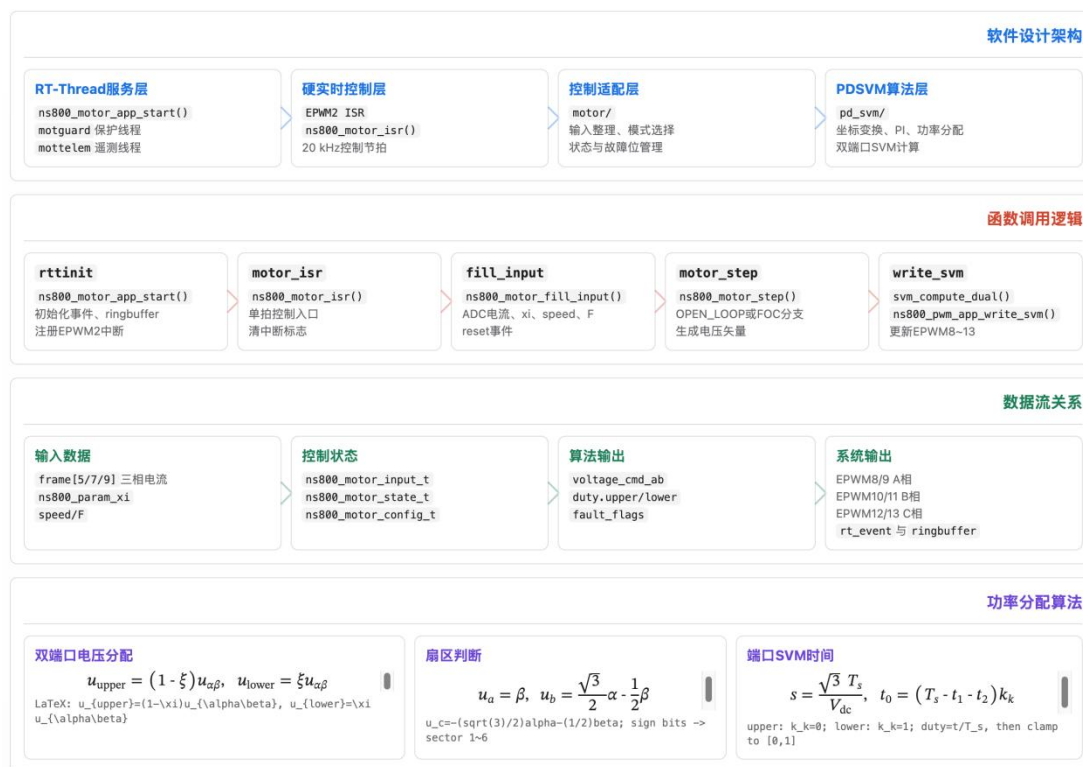
初始化流程: ns800_app_main() 调用 ns800_display_app_start() 创建 oled 线程。线程入口 ns800_display_thread_entry() 首先调用 ns800_sh1106_oled_start(), 再由 OLED 驱动启动 ns800_i2c1_app_start()。I2C1 初始化包括创建 i2c1 mutex、配置 GPIO91/92 为 SDA/SCL、设置 400 kHz 主机波特率并使能 I2C 主机模块。

显示刷新流程： OLED 线程启动后进入周期循环，每 100 ms 调用 ns800_draw_status_page()。该函数读取 ADC 最新采样帧，计算 HV/LV 功率和三相总功率；同时读取 ns800_motor_app_get_status() 获取电机输出功率，并读取按键模块维护的 xi/speed/F 参数，最终写入 SH1106 framebuffer 并刷新屏幕。

线程同步机制：I2C1 写操作集中封装在 `ns800_i2c1_app_write()` 中。每次发送前调用 `rt_mutex_take(i2c1_lock, RT_WAITING_FOREVER)`，发送完成后释放互斥量。当前主要由 OLED 线程使用 I2C1，但互斥锁为后续扩展多个 I2C 外设提供总线级保护，防止多个线程同时发起 I2C 传输。

跨模块数据关系: OLED 显示线程是只读消费者: 从 ADC-DMA 环形缓存读取采样数据, 从电机应用状态读取控制输出, 从按键线程读取用户设定参数。它不反向修改控制状态, 也不进入 ISR 快路径。该设计将显示刷新与实时控制解耦, 保证界面更新可见, 同时不引入控制周期抖动。

4) 功率分配 PDSVM 算法与控制系统实现:



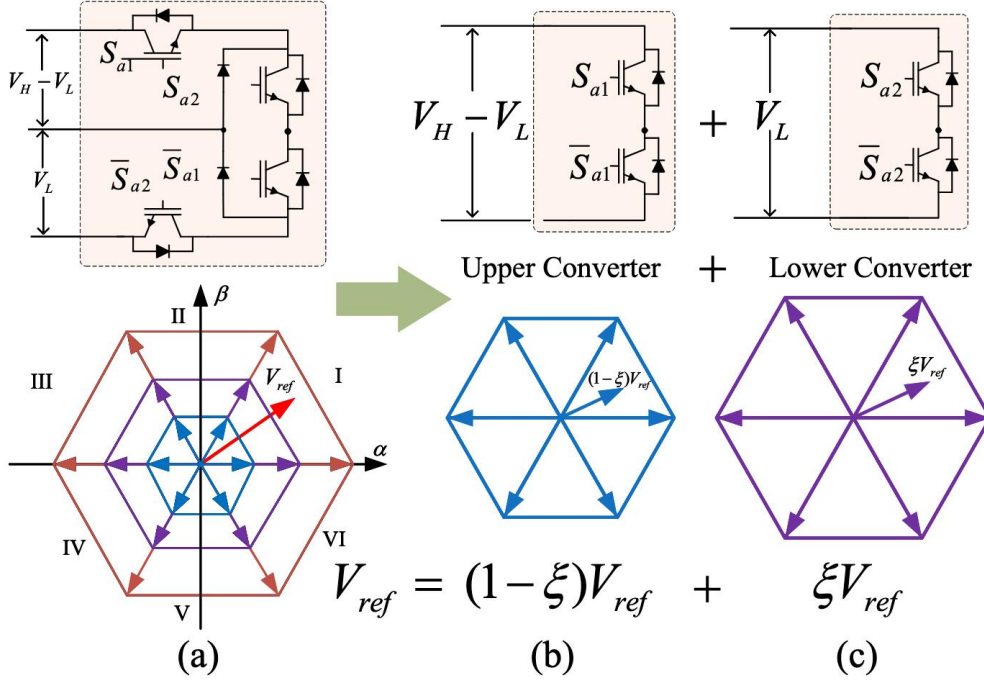
该部分采用“RT-Thread 系统服务 + EPWM2 硬实时控制 ISR + PDSVM 算法库 + EPWM8~13 功率输出”的分层结构。RT-Thread 负责 ns800_motor_app_start() 启动、motguard 故障保护线程、mottelem 遥测线程和 FinSH 调试命令；EPWM2 ISR 以 20 kHz 执行控制快路径；motor/ 完成控制状态管理；pd_svm/ 完成坐标变换、PI、功率分配和双端口 SVM 占空比计算。

函数调用逻辑：系统启动时，rttinit 线程调用 ns800_motor_app_start()，初始化电机参数、控制状态、RT-Thread 事件和遥测 ringbuffer，然后注册 ns800_motor_isr() 到 EPWM2 中断。运行时 ISR 依次调用 ns800_motor_fill_input()、ns800_motor_step()、ns800_pwm_app_write_svm()。其中 ns800_motor_step() 在 OPEN_LOOP 模式下生成旋转电压矢量，在闭环模式下保留 Clarke/Park、速度 PI、电流 PI 和解耦前馈流程。

数据流关系：控制输入来自三类数据：ADC-DMA 最新帧提供三相电流，按键线程提供 xi/speed/F 参数，反馈接口预留机械角和速度。控制输出为 ns800_motor_output_t，包含电压指令、功率估计、双端口占空比和状态位。占空比直接写入 EPWM8~13；故障状态通过 rt_event 发送给 motguard；降采样遥

测通过 `rt_ringbuffer` 发送给 `mottelem`。因此实时控制和线程服务只通过轻量数据边界耦合。

功率分配算法： PDSVM 先按功率分配系数 ξ 拆分同一个电压矢量：



$$\mathbf{u}_{upper} = (1 - \xi)\mathbf{u}_{\alpha\beta}, \quad \mathbf{u}_{lower} = \xi\mathbf{u}_{\alpha\beta}, \quad 0 \leq \xi \leq 1$$

随后 upper/lower 分别调用端口 SVM。扇区判断采用

$$u_a = \beta, \quad u_b = \frac{\sqrt{3}}{2}\alpha - \frac{1}{2}\beta, \quad u_c = -\frac{\sqrt{3}}{2}\alpha - \frac{1}{2}\beta$$

端口 SVM 按扇区计算有效矢量时间，并用

$$s = \frac{\sqrt{3}T_s}{V_{dc}}, \quad t_0 = (T_s - t_1 - t_2)k_k$$

生成占空比。

$$\mathbf{d}_U = \begin{bmatrix} \tau_{1U} + \tau_{2U} \\ \tau_{2U} \\ 0 \end{bmatrix} \quad \mathbf{d}_L = \begin{bmatrix} \tau_{1L} + \tau_{2L} + \tau_{0L} \\ \tau_{2L} + \tau_{0L} \\ \tau_{0L} \end{bmatrix}.$$

代码中 upper 端口 $kk=0$ ，lower 端口 $kk=1$ ，两端口零矢量偏置不同，最终形成六组占空比并映射到 EPWM8/9、EPWM10/11、EPWM12/13。

功率分配情况如下所示：

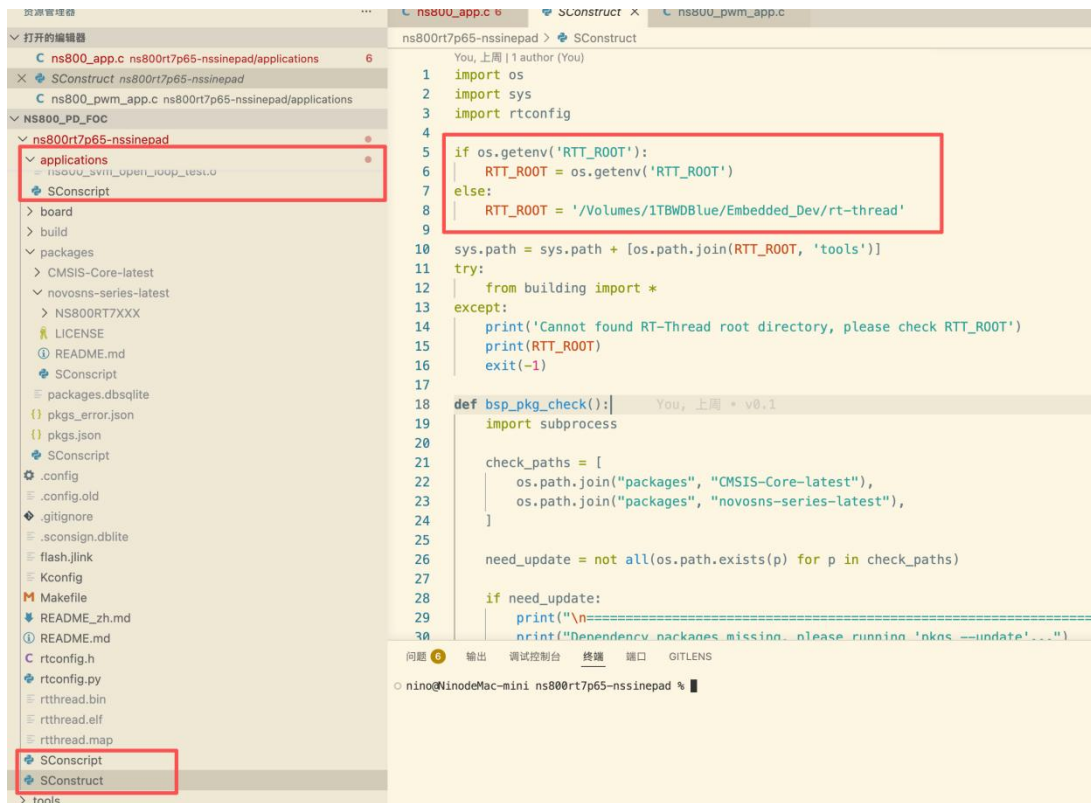
$$\mathbf{P}_{dc} = \begin{bmatrix} P_H \\ P_L \end{bmatrix} = \begin{bmatrix} (1 - \xi) \frac{V_H}{V_H - V_L} \\ \xi \left(1 + \frac{V_L}{V_H - V_L} \right) - \frac{V_L}{V_H - V_L} \end{bmatrix} P_{ref}$$

5) RT-Thread 适配编译脚本(基于 SCons 工具):

本工程采用 RT-Thread 标准 SCons 构建体系。顶层 SConstruct 负责确定 RTT_ROOT、加载 tools/building.py，并通过 PrepareBuilding() 引入 RT-Thread 内核、组件和 BSP 构建规则。编译前脚本会检查 CMSIS-Core-latest 与 novosns-series-latest 是否存在，缺失时提示执行 `pkgs --update`。

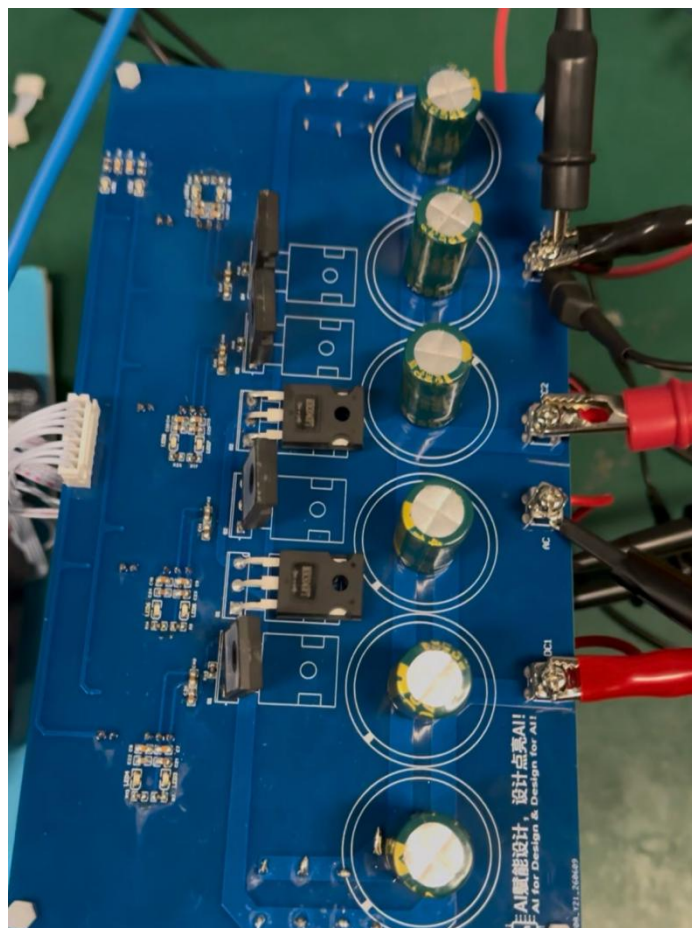
`applications/SConscript` 递归收集 `applications/` 下的 .c 文件，并自动加入头文件路径，因此 PDSVM、电机控制、ADC、OLED、按键等模块可直接纳入构建。

`libraries/HAL_Drivers/drivers/SConscript` 根据 `rtconfig.h` 中的宏选择驱动源码，例如 `BSP_USING_GPIO`、`BSP_USING_UART`、`BSP_USING_ADC`、`BSP_USING_CAN`。这些宏同时决定驱动是否通过 RT-Thread 设备框架注册。整体上，SCons 脚本实现了“RT-Thread 内核 + NOVOSNS 驱动 + 应用功能模块”的统一构建与可配置裁剪。



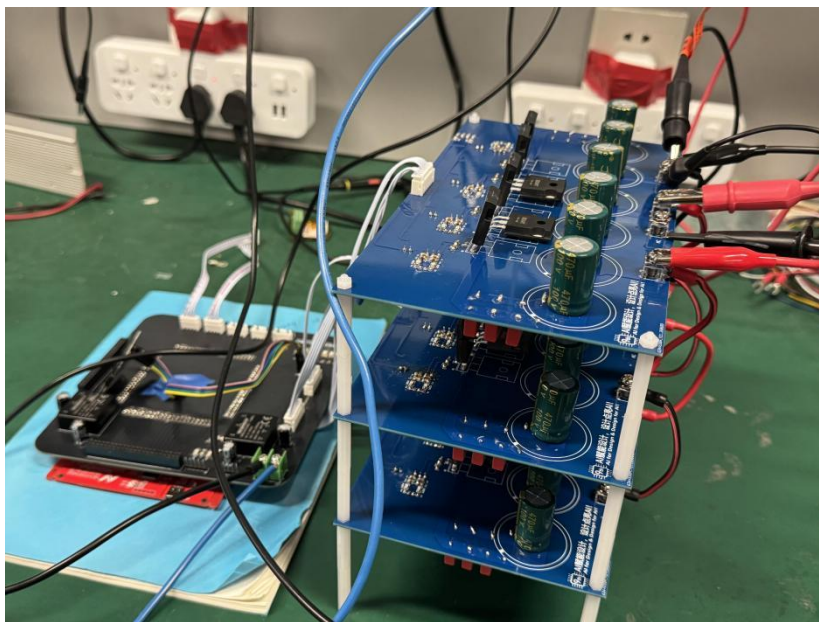
第三部分 完成情况及性能参数

3.1 整体介绍



如图，展示的是单级式多端口 NPC 型三电平拓扑逆变器。其中 DC1 端口连接高压直流端口，DC2 端口连接低压直流端口，构成多端口特征。AC 为逆变器交流输出，连接负载。

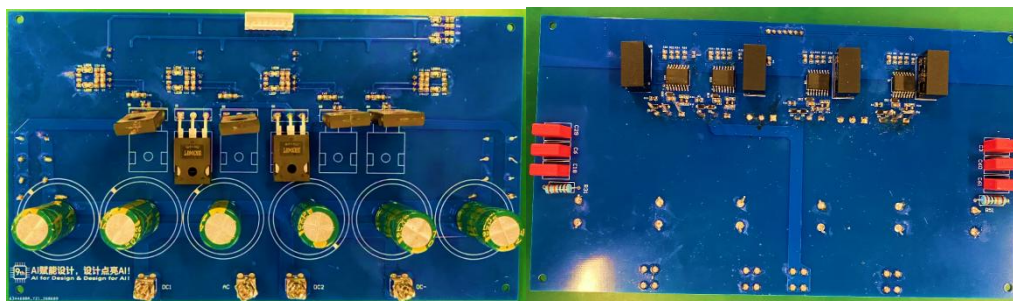
每相桥臂的开关管由 4 个 IGBT 与 2 个二极管组成，构成 NPC 型三电平拓扑结构。



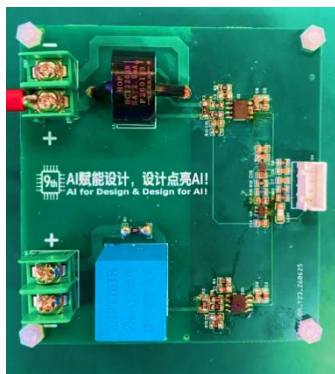
如图，展示的是数字系统控制板与三相桥臂的连接与系统整体构成。左侧为纳芯微 NS800 开发板及其拓展板，右侧为三相桥臂。

3.2 工程成果

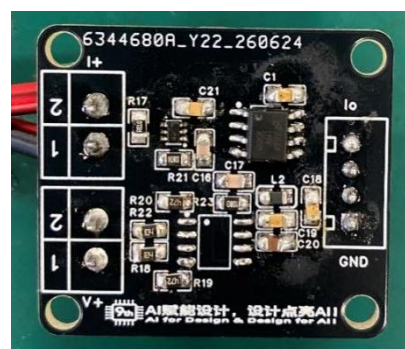
3.2.1 电路成果；



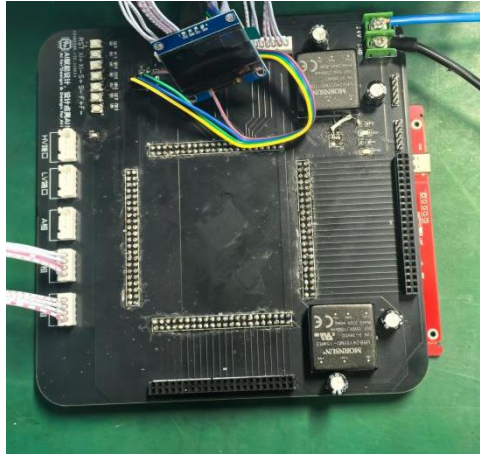
三电平单级多端口逆变器



交流电压电流采样



直流电压电流采样



拓展板

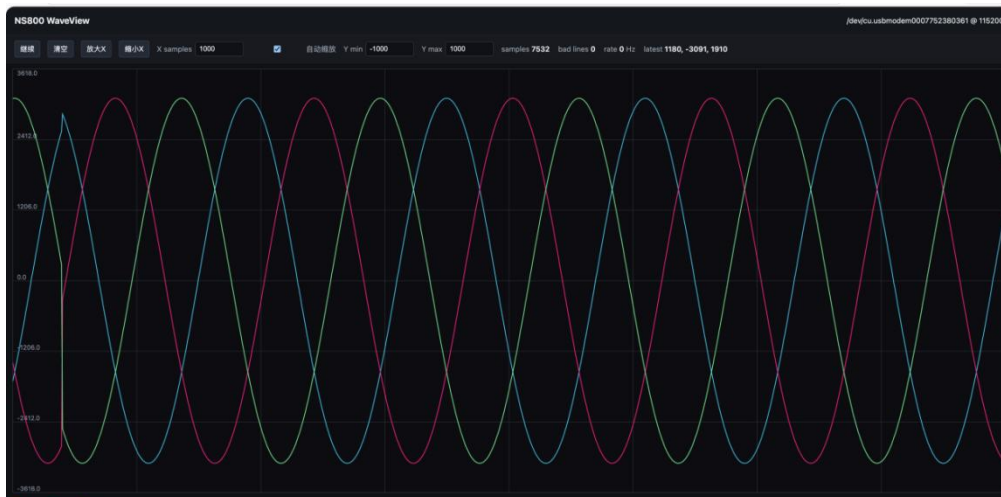
3.2.2 软件成果;

1) 功率分配 PDSVM 的 AC 侧输出波形 (基于占空比进行独立合成)

```

142 // ol_status.last_duty.lower.tc * ol_status.lvdc_v) * NS800_SVM_UL_PKINI_SCALE);
143
144 float va_f = ol_status.last_duty.upper.ta * ol_status.hvdc_v - ol_status.last_duty.lower.ta * ol_status.l
145 float vb_f = ol_status.last_duty.upper.tb * ol_status.hvdc_v - ol_status.last_duty.lower.tb * ol_status.l
146 float vc_f = ol_status.last_duty.upper.tc * ol_status.hvdc_v - ol_status.last_duty.lower.tc * ol_status.l
147
148 float vcm = (va_f + vb_f + vc_f) / 3.0f;
149
150 va = (va_f - vcm) * 1000;
151 vb = (vb_f - vcm) * 1000;
152 vc = (vc_f - vcm) * 1000;
153
154 rt_kprintf("%d,%d,%d\r\n", va, vb, vc);
155
156 rt_thread_mdelay(1);
157

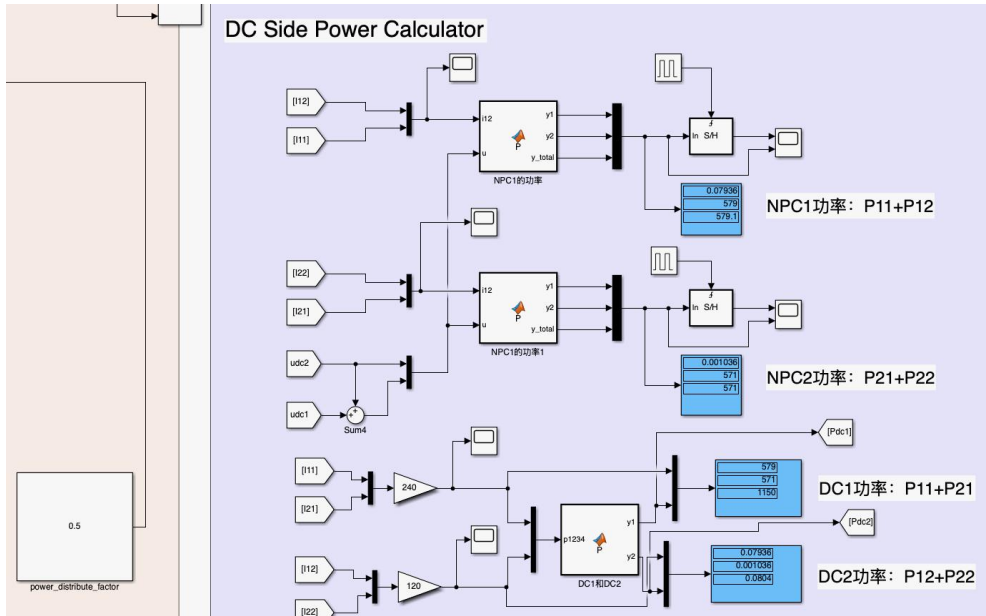
```



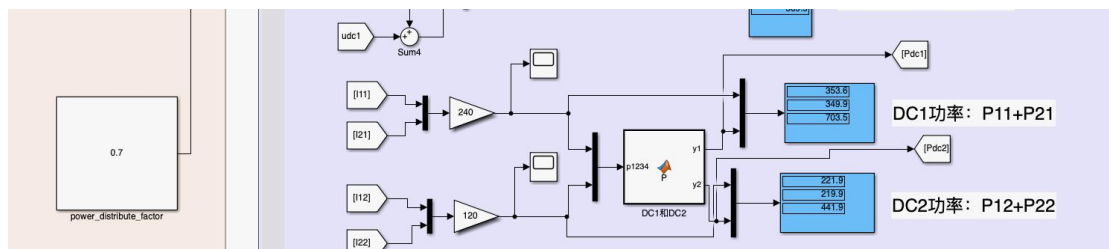
2) 功率分配功能实现:

为便于测试与观测，算法的验证在 simulink 中进行。

在电压比 $r=0.5$ 条件下，功率分配系数设定为 0.5 时，低压端口输出功率为 0，此时高压端口承担全部负载输出功率。

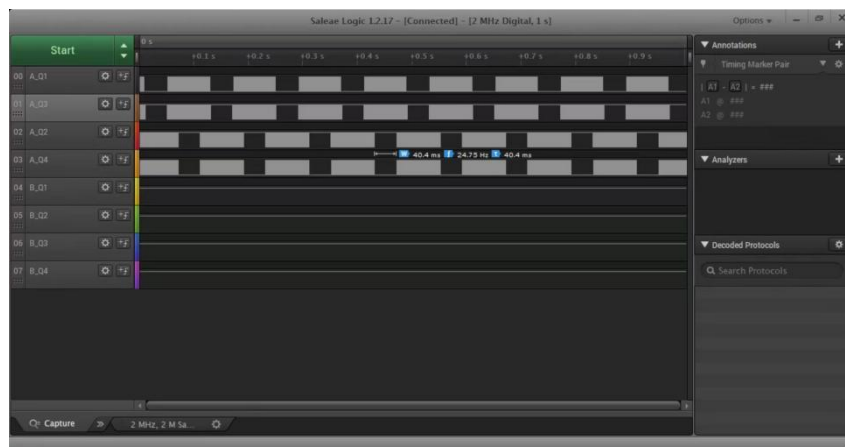


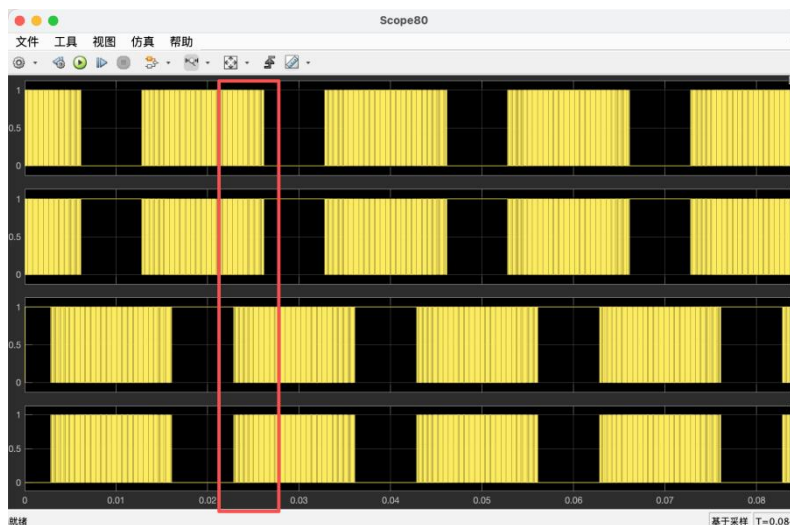
将功率分配系数设定为 0.7，此时低压端口开始承担功率输出份额



3) 逻辑分析仪实测 PWM 波形与 Simulink 算法仿真对比:

对 A 相桥臂进行分析，波形特征一致。

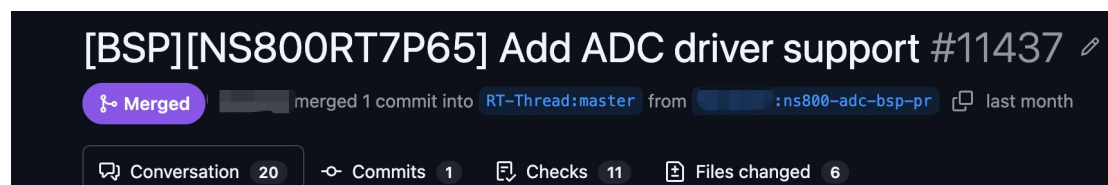




4) OLED 实时显示 ADC 采样后功率解算值

5) 基于 RT-Thread 的 NS800 可复用开发

开发并适配了 NS800 芯片的 RT-Thread ADC 外设，已提交 PR 并 Merge 入主线



3.3 特性成果

软件部分：

1.RT-Thread 线程间通信

以按键参数调节为例，应用层先构造一次虚拟按键动作，例如调节功率分配系数、目标转速或力矩电流。生产端将按键编号、动作类型和来源信息封装成消息，通过 RT-Thread 消息队列发送出去；同时释放信号量，通知消费侧“有新的按键动作需要处理”。随后事件集置位，表示本次按键处理窗口已经打开。

消费侧先通过 `rt_event_recv()` 捕获窗口事件，确认当前只处理本次触发的数据；再通过 `rt_sem_take()` 获取按键触发信号，最后用 `rt_mq_recv()` 读取具体按键动作。拿到动作后，应用层调用参数处理函数，完成 `xi`、转速、力矩电流或复位计数的更新。

这个流程体现了 RT-Thread IPC 组合使用的优势：事件集负责流程边界，信号量负责触发同步，消息队列负责传递结构化数据。三者职责清晰，使按键扫描、业务处理和参数更新天然解耦，也方便通过串口 trace 观察“输入产生、信号触发、消息消费、参数变化”的完整链路。

2. 结合 RTThread 线程通信与同步锁的电机数据遥感测量能力

遥测链路展示 RT-Thread 对连续数据的处理能力。控制 ISR 产生运行帧后写入 ringbuffer，再通过事件集通知消费者。消费者收到事件后批量读取 ringbuffer，输出 ISR 计数、ADC 序号、故障位和控制参数。这样既避免 ISR 里直接打印阻塞实时控制，又能用窗口式串口输出观察关键运行路径。RT-Thread 的 IPC 和 ringbuffer 组合，让实时侧与展示侧自然解耦。

```
msh >wb_telem

[flow][motor-telem] begin

[flow][virtual-motor-isr] -> ringbuffer telem isr=120 adc_seq=9

[flow][virtual-motor-isr] -> ringbuffer telem isr=220 adc_seq=10

[flow][virtual-motor-isr] -> event:TELEM_READY

[flow][motor-telem] call consumer window directly

[flow][telem-consumer] <- event:TELEM_READY bits=0x00000040

[flow][telem-consumer] <- ringbuffer frame isr=120 adc_seq=9 fault=0x00000000 xi=500/1000

[flow][telem-consumer] <- ringbuffer frame isr=220 adc_seq=10 fault=0x00000001 xi=450/1000

[flow][telem-consumer] -> event:TELEM_DRAINED count=2

[flow][motor-telem] <- event:TELEM_DRAINED bits=0x00000080

[flow][motor-telem] end consumed=2

msh >
```

3. 电机守护进程捕获到故障后即刻停机

以电机故障处理为例，控制 ISR 检测到低压、采样异常等状态后，将故障信息封装为消息，通过 RT-Thread 消息队列传递给保护逻辑；同时用事件集通

知“故障已发生”。保护逻辑通过 `rt_event_recv()` 捕获事件，再用 `rt_mq_recv()` 读取故障来源、标志位和采样序号，随后通过邮箱发送 STOP 动作。执行端用 `rt_mb_recv()` 获取停机命令，完成 PWM 输出关闭。这个链路体现了 RT-Thread 事件通知、消息传参、邮箱命令分发的清晰分层。

```

wb_fault

[flow][motor-fault] begin

[flow][virtual-motor-isr] -> mq:fault_msg isr=33 fault=0x00000003

[flow][virtual-motor-isr] -> event:MOTOR_FAULT

[flow][motor-fault] call guard window directly

[flow][motor-guard] <- event:MOTOR_FAULT bits=0x00000010

[flow][motor-guard] <- mq:fault_msg isr=33 fault=0x00000003

[flow][motor-guard] -> mailbox:STOP

[flow][motor-fault] call actuator window directly

[flow][motor-actuator] <- mailbox action=1

[flow][motor-actuator] -> virtual pwm stop, stop_count=1

[flow][motor-fault] <- event:MOTOR_STOP bits=0x00000020

[flow][motor-fault] end fault=0x00000003 stop_count=1

msh >
  
```

4.ADC 采样与数据流传输

以 ADC 数据同步为例，采样侧先生成一帧虚拟 DMA 数据，包含序号和多路原始 ADC 通道值。数据进入同步链路后，生产端通过 `rt_mq_send()` 将完整采样帧放入消息队列，并通过 `rt_event_send()` 置位 `ADC_FRAME_READY`，表示新数据已准备好。

消费侧由电机输入处理逻辑接管，先调用 `rt_event_recv()` 捕获帧就绪事件，再通过 `rt_mq_recv()` 取出 ADC 帧。随后使用 `rt_mutex_take()` 进入受保护区域，将原始采样值转换为 A/B/C 三相电流；转换完成后 `rt_mutex_release()` 释放互

斥锁，并通过事件通知同步完成。

这个流程体现了 RT-Thread 在数据同步场景中的优势：事件集负责“什么时候处理”，消息队列负责“处理什么数据”，互斥锁保证共享结果更新的一致性。三类内核机制分工明确，使采样生产、数据搬运和控制输入更新解耦，既贴近真实 ADC DMA 工作方式，又方便用串口 trace 清楚观察每一步数据流。

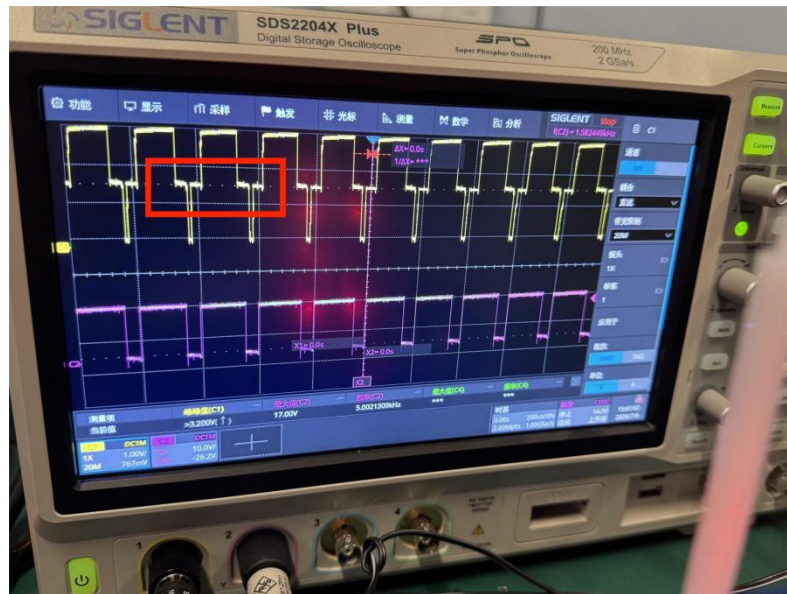
```
msh >wb_adc

[flow][adc] begin
[flow][virtual-adc-dma] -> mq:adc_frame seq=7
[flow][virtual-adc-dma] -> event:ADC_FRAME_READY
[flow][adc] call consumer window directly
[flow][adc-consumer] <- event:ADC_FRAME_READY bits=0x00000004
[flow][adc-consumer] <- mq:adc_frame seq=7 raw5=2100 raw7=2000 raw9=2058
[flow][motor-input] -> mutex:latest_adc_lock
[flow][motor-input] <- ia=52 mA ib=-48 mA ic=10 mA
[flow][adc-consumer] -> event:ADC_SYNC_DONE
[flow][adc] <- event:ADC_SYNC_DONE bits=0x00000008
[flow][adc] end seq=7
```

硬件部分：

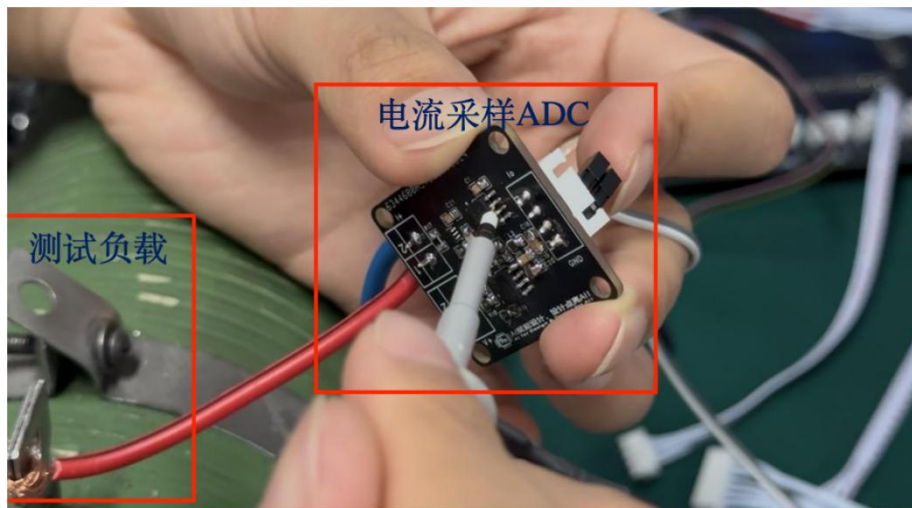
1.三电平逆变器拓扑

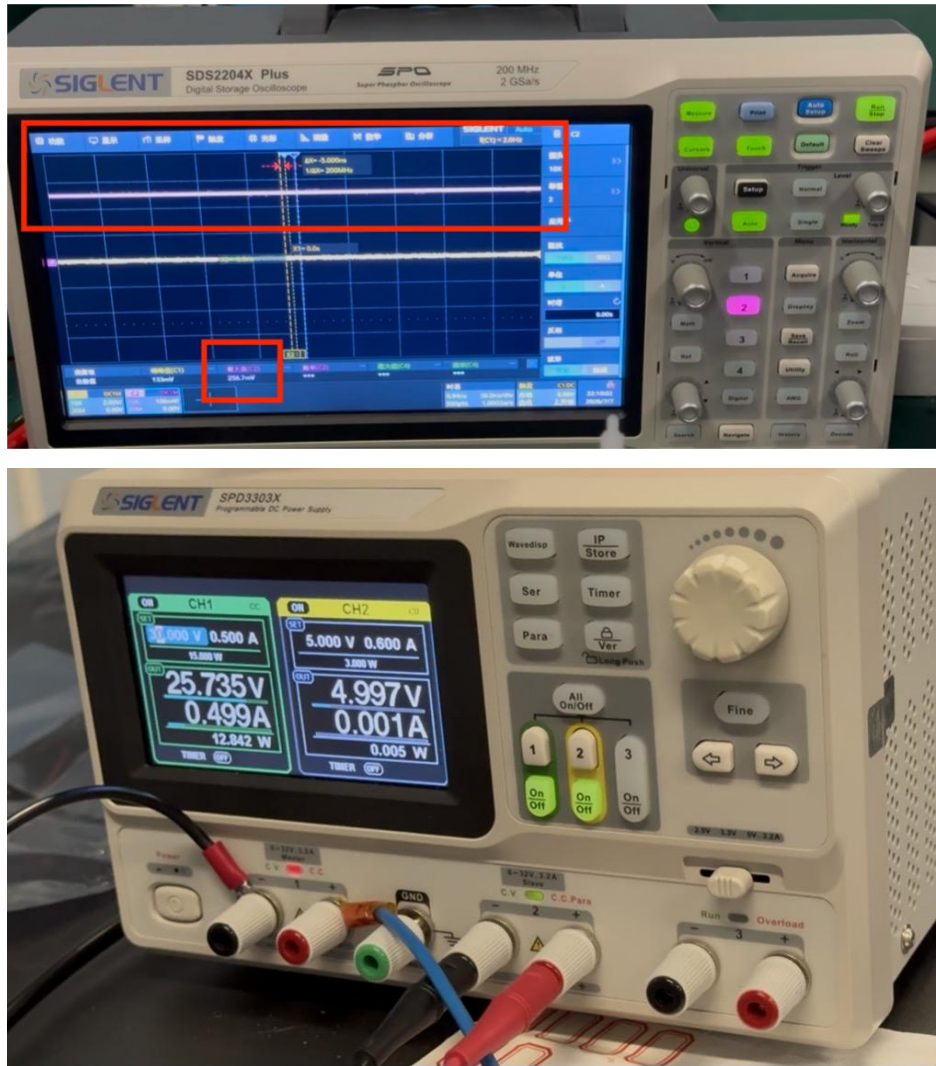
示波器开关管波形实测--三电平逆变器结构下的开关管波形，清晰呈现出三电平拓扑的中间态电平。



2.ADC 采样效果

这里展示了 ADC 采样电路（电流采样）的换算能力，ADC 理论测量范围上限为 6.6A，此时 250mV 左右对应 500mA 输出电流，实测符合预期。





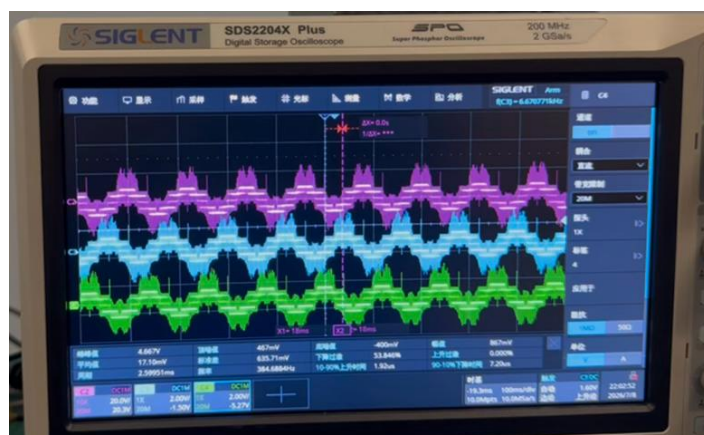
3.带载能力与功率分配能力

驱动三相电阻负载：

每个电阻的阻值为 50 欧姆，星形（Y 型）连接。



示波器探头夹取负载中性点与三相电阻接入逆变器输出的节点

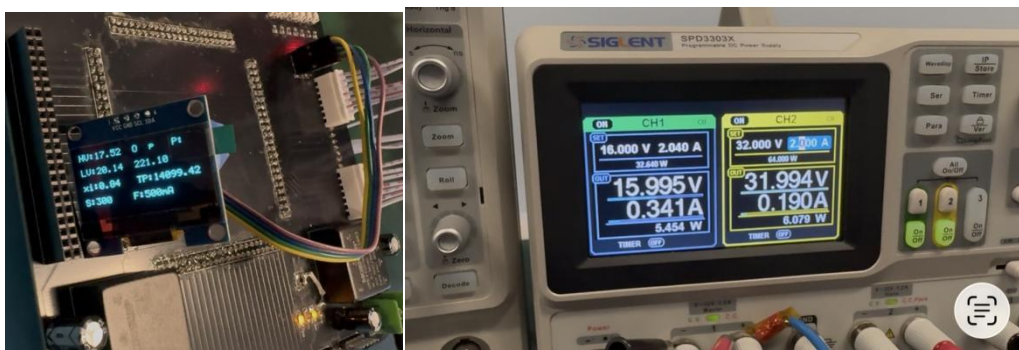


此时的波形呈现出清晰的三电平三相逆变输出特征（无电感滤波）。

在 $\xi=0.73$ 时，双 DC 端口的功率输出如图所示

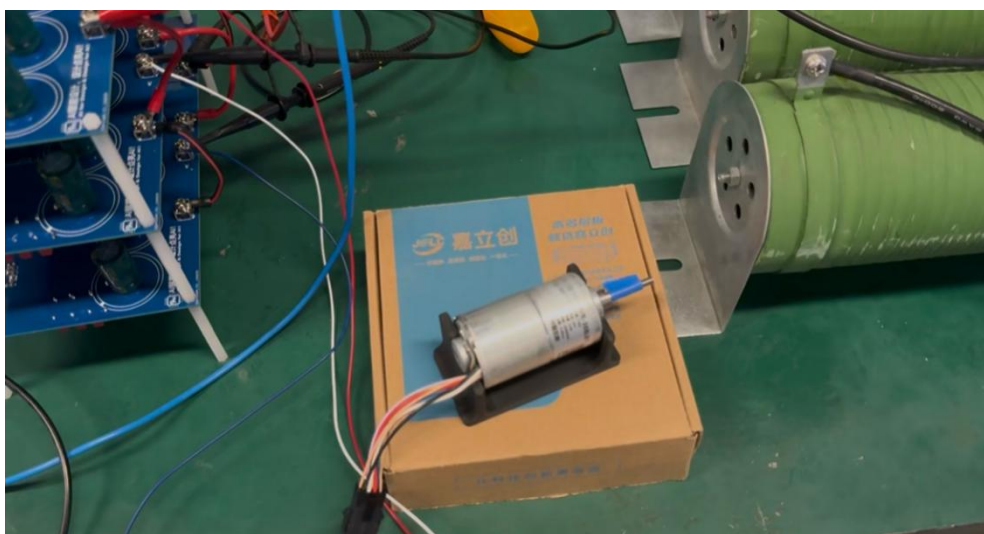


在 $\xi=0.84$ 时，双 DC 端口的功率输出如图所示：

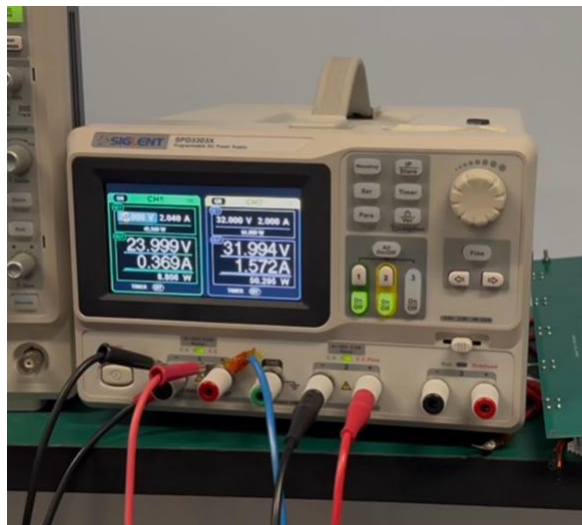


呈现出了精确的功率分配能力。

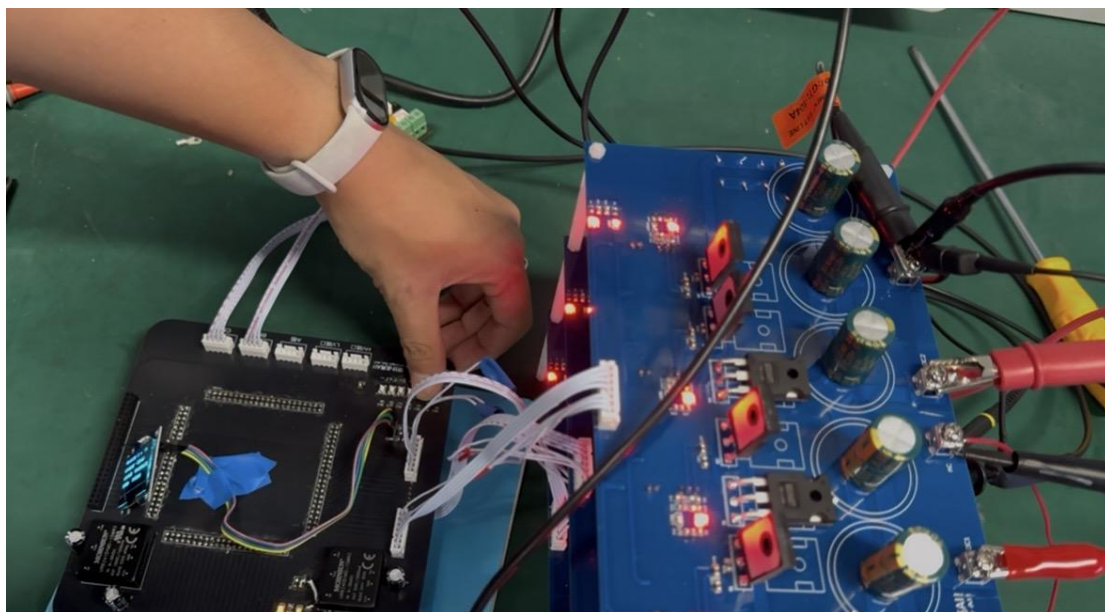
驱动三相无刷电机负载：



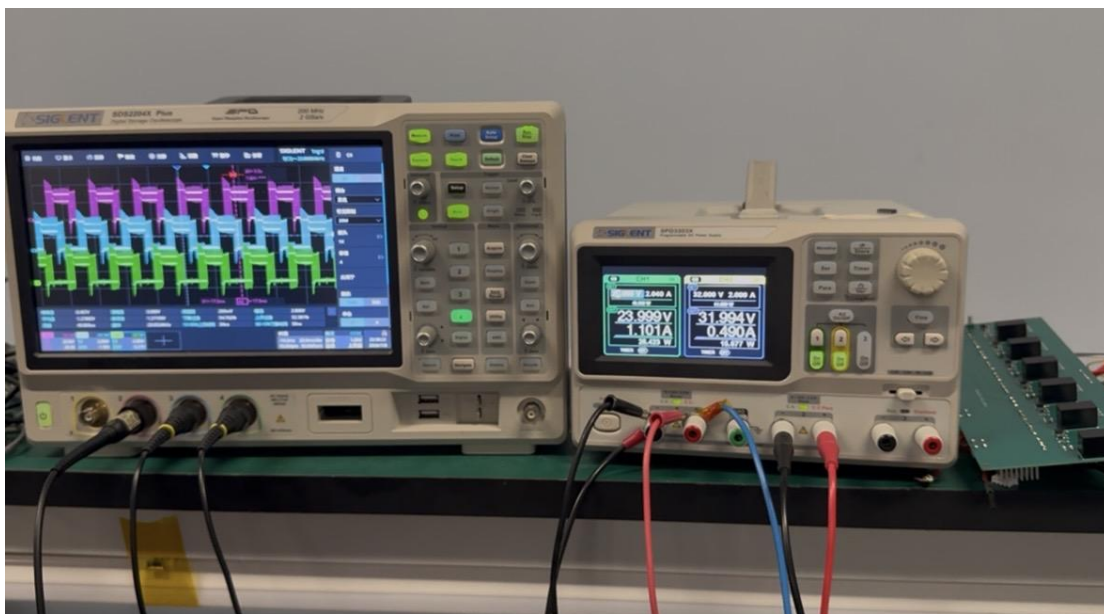
电机负载的功率需求大约为 50W, 在默认参数 $\xi=0.63$ 下, 低压端口 DC2 (24V) 不参与电机负载侧的功率输出。可以看出, 此时的低压端口 DC2 (直流稳压电源 Channel 1) 仅为数字系统提供功率输出, 功率约为 7W。此时的高压端口 DC1 (直流稳压电源 Channel 2) 为负载提供全部功率输出, 功率约为 50W。



调节功率分配系数 ξ



低压端口开始承担更多的功率输出份额，此时的低压端口输出功率达到了24W，高压端口也同步继续输出功率，实现了功率分配的灵活调整。



第四部分 总结

4.1 可扩展之处

本工程已形成较清晰的 RT-Thread 分层和硬实时控制框架，后续可从三方面扩展。第一，控制闭环可继续完善，将当前预留的机械角/速度反馈接口替换为编码器、霍尔或无感观测器输出，实现速度环和电流环闭环运行。第二，PDSVM 算法可扩展为在线功率分配策略，使 x_i 不再只由按键给定，而是根据 HV/LV 端口电压、电流、功率或温升自动调整。第三，RT-Thread 服务层可继续增强，包括 CANFD 通信、故障记录、参数掉电保存、远程调试和多页面 OLED 显示，使系统从功能验证平台逐步演进为完整电机驱动控制器。

4.2 心得体会

本次软件设计的核心体会是：在电机控制这类强实时系统中，RTOS 并不是用来替代硬实时控制环，而是用于组织控制环之外的系统服务。工程实现中，EPWM2 ISR 负责 20 kHz 控制周期，EPWM1、ADCA 和 EDMA 负责连续采样，EPWM8~13 负责 PWM 输出，这些关键路径必须保持严格确定性；而 RT-Thread 更适合承担线程调度、系统启动、Shell 调试、OLED 显示、故障处理和遥测输出等任务。通过这种分层设计，可以避免复杂逻辑进入中断，同时防止线程调度影响控制周期，提高系统稳定性和可维护性。

开发过程中，一个重要经验是明确系统数据流边界。ADC-DMA 模块仅负责更新采样帧，控制 ISR 读取当前周期所需的三相电流和母线信息，OLED 线程只访问显示变量。按键线程通过共享变量提供 x_i 、speed、F 等参数，故障处理通过 `rt_event` 通知保护线程，遥测数据通过 `rt_ringbuffer` 从 ISR 传递至日志线程。不同数据根据实时性需求采用不同通信方式，而非统一使用单一 IPC 机制，更符合嵌入式实时系统设计原则。

PDSVM 算法从仿真模型迁移到嵌入式实现时，也需要考虑大量工程因素。算法首先根据电压分配系数 x_i 将目标电压矢量分配到 upper/lower 端口，再分别计算两个端口的 SVM 占空比；实际实现中还需要处理母线电压变化、调制范围

限制、过调制、占空比钳位以及 PWM 寄存器映射等问题。最终通过统一接口，使 `duty.upper` 和 `duty.lower` 能够直接对应 EPWM8~13 输出，实现控制算法与功率桥臂之间的直接映射。

系统开发初期，基于 NS800RT7P65 BSP 工程搭建 RT-Thread 环境，完成 SCons 构建配置、`rtconfig.h` 参数调整以及外设驱动适配。随后按照由底层到上层的方式开发：首先验证 EPWM 输出和 ADC-DMA 采样链路，再加入按键调节和 OLED 显示，最后将电机控制算法和 PDSVM 功率分配算法接入 EPWM2 中断，实现完整控制流程。

开发过程中也遇到了一些典型问题。例如，在 PWM 调试阶段曾出现输出相位异常，通过逐路验证 EPWM 配置、关闭同步功能并结合示波器观察波形，最终定位到 PWM 同步和寄存器配置问题。在 ADC-DMA 调试中，曾出现采样数据错位和控制读取异常，通过检查 DMA 地址、ADC 触发源以及数据更新时间，发现问题来自采样帧更新与 ISR 读取不同步，因此采用固定采样帧结构，并保证 ISR 读取完整数据，提高采样可靠性。

在 RTOS 集成过程中，OLED 刷新、Shell 调试等任务执行时间较长，若直接放入控制中断，会造成控制周期抖动。因此将显示、日志和按键扫描任务全部放入 RT-Thread 线程中，并通过共享变量、事件通知和环形缓冲区通信，使 20 kHz 控制环保持稳定运行。同时采用逐步增加任务模块的方法验证系统负载，降低调试复杂度。

最终调试时，先通过开环旋转电压矢量验证 EPWM8~13 输出关系，再检查 ADC 采样稳定性，随后接入控制算法并观察执行时间、PWM 占空比和系统状态变化，逐步排查算法、采样和硬件映射问题。通过加入 `rt_event` 和 `rt_ringbuffer`，实现故障保护和遥测输出，提高了系统状态监控和问题定位能力。

通过本次设计，我认识到复杂嵌入式控制系统的关键不仅在于控制算法本身，更在于实时控制、外设驱动、任务调度和系统服务之间的合理组织。可靠的电机控制系统需要明确硬实时路径和软实时服务的边界，通过合理的软件架构保证控制性能，同时提升系统扩展性和工程维护效率。此次开发也进一步加深了我对 RTOS 在实时控制系统中的应用方式，以及理论算法向实际嵌入式实现过程中工

程细节重要性的理解。

第五部分 参考文献

- [1] Z. Huang, D. Zhou, L. Wang, Z. Shen, and Y. Li, “A review of single-stage multiport inverters for multisource applications,” *IEEE Trans. Power Electron.*, vol. 38, no. 5, pp. 6566 – 6584, May 2023, doi: 10.1109/TPEL.2023.3234358.
- [2] Z. Huang, D. Zhou, Z. Shen, and J. Zou, “Directed graph-based topology derivation method for single-stage multiport inverters,” *IEEE Trans. Power Electron.*, vol. 38, no. 11, pp. 14614 – 14627, Nov. 2023, doi: 10.1109/TPEL.2023.3305988.
- [3] L. Liu, D. Zhou, J. Zou, and W. Wang, “Decoupled modeling and wide-range power distribution strategy for the multisource inverter in microgrids,” *IEEE Trans. Power Electron.*, vol. 38, no. 10, pp. 12078 – 12090, Oct. 2023, doi: 10.1109/TPEL.2023.3285008.
- [4] D. Zhou, L. Liu, Y. Mao, and J. Zou, “Asymmetrical level-shifted PWM-based power distribution control of single-stage multiport inverter-connected islanded microgrids,” *IEEE Trans. Power Electron.*, vol. 39, no. 10, pp. 12294 – 12305, Oct. 2024, doi: 10.1109/TPEL.2024.3404277.
- [5] L. Liu, D. Zhou, J. Zou, and Z. Shen, “Direct power distribution strategy based on space vector modulation for single-stage dual-DC-port inverter,” *IEEE Trans. Power Electron.*, vol. 40, no. 8, pp. 11552 – 11563, Aug. 2025, doi: 10.1109/TPEL.2025.3555664.
- [6] Y. Huang, D. Zhou, X. Liu, J. Zou, and Z. Shen, “Hybrid SVM with duty cycle waveform split for power allocation of single-stage multiport inverter-formed islanded microgrids,” *IEEE Trans. Power Electron.*, vol. 41, no. 3, pp. 4260 – 4271, Mar. 2026, doi: 10.1109/TPEL.2025.3616128.
- [7] W. Zhang, D. Zhou, Z. Shen, J. Zou, and X. Liu, “Dual synchronous rotating frame-based power allocation control of single-stage multiport inverter in islanded microgrids,” *IEEE Trans. Power Electron.*, vol. 41, no. 1, pp. 1067 – 1076, Jan. 2026, doi: 10.1109/TPEL.2025.3603997.
- [8] K. Luo, D. Zhou, Z. Zhang, J. Zou, and X. Zhou, “Three-dimensional vector cube

- decomposition-based power allocation for multisource inverter-fed PMSM drives,” IEEE Trans. Power Electron., vol. 40, no. 8, pp. 10705 – 10716, Aug. 2025, doi: 10.1109/TPEL.2025.3554334.
- [9] D. Zhou, Y. Ding, S. Jia, and J. Zou, “Hybrid predictive control with linear power distribution of single-stage multiport inverters for hybrid electric vehicles,” IEEE Trans. Ind. Electron., vol. 72, no. 11, pp. 11181 – 11190, Nov. 2025, doi: 10.1109/TIE.2025.3563726.
- [10] D. Zhou, Y. Mao, and J. Zou, “Double zero-sequence voltage injection-based power distribution control of single-stage multiport inverter-fed hybrid electric vehicles,” IEEE Trans. Ind. Electron., vol. 72, no. 2, pp. 1347 – 1356, Feb. 2025, doi: 10.1109/TIE.2024.3426109.
- [11] D. Zhou, K. Luo, Z. Shen, and J. Zou, “Vector-space-decomposition-based power flow control of single-stage-multiport-inverter-fed PMSM drive for hybrid electric vehicles,” IEEE Trans. Ind. Electron., vol. 71, no. 8, pp. 8514 – 8524, Aug. 2024, doi: 10.1109/TIE.2023.3322004.
- [12] Y. Yu, S. Jia, X. Liu, J. Zou, and D. Zhou, “Iterative learning-based power distribution control of multisource inverters via zero-sequence component compensation,” IEEE Trans. Ind. Electron., vol. 72, no. 9, pp. 9128 – 9138, Sep. 2025, doi: 10.1109/TIE.2025.3539327.
- [13] Y. Yu, D. Zhou, Z. Shen, J. Zou, and X. Liu, “Repetitive control-based power distribution scheme for single-stage dual-port inverters with power ripple reduction,” IEEE Trans. Ind. Electron., vol. 73, no. 4, pp. 5603 – 5614, Apr. 2026, doi: 10.1109/TIE.2025.3629319.